

Graphs for Uncertainty-Aware Code Generation and Verification

22AIE301 Course Project — Review 1

Abdul Azhar • CB.SC.U4AIE24301

Deepak Prabhu • CB.SC.U4AIE24312

Pranesh M • CB.SC.U4AIE24345

Sandhiya D • CB.SC.U4AIE24353

Outline of the Presentation

- Introduction
- Literature Review
- Objectives
- Methodology
- Progress in terms of Project Objectives
- Weekly Timeline for Project completion
- References

Introduction

- AI-written code often *looks* correct but can hide bugs.
- Current checks only say pass/fail — not *which part* to worry about.
- Our idea: give every small piece of code its own trust score.
- **Goal: show exactly which code needs a human look.**

Literature Review

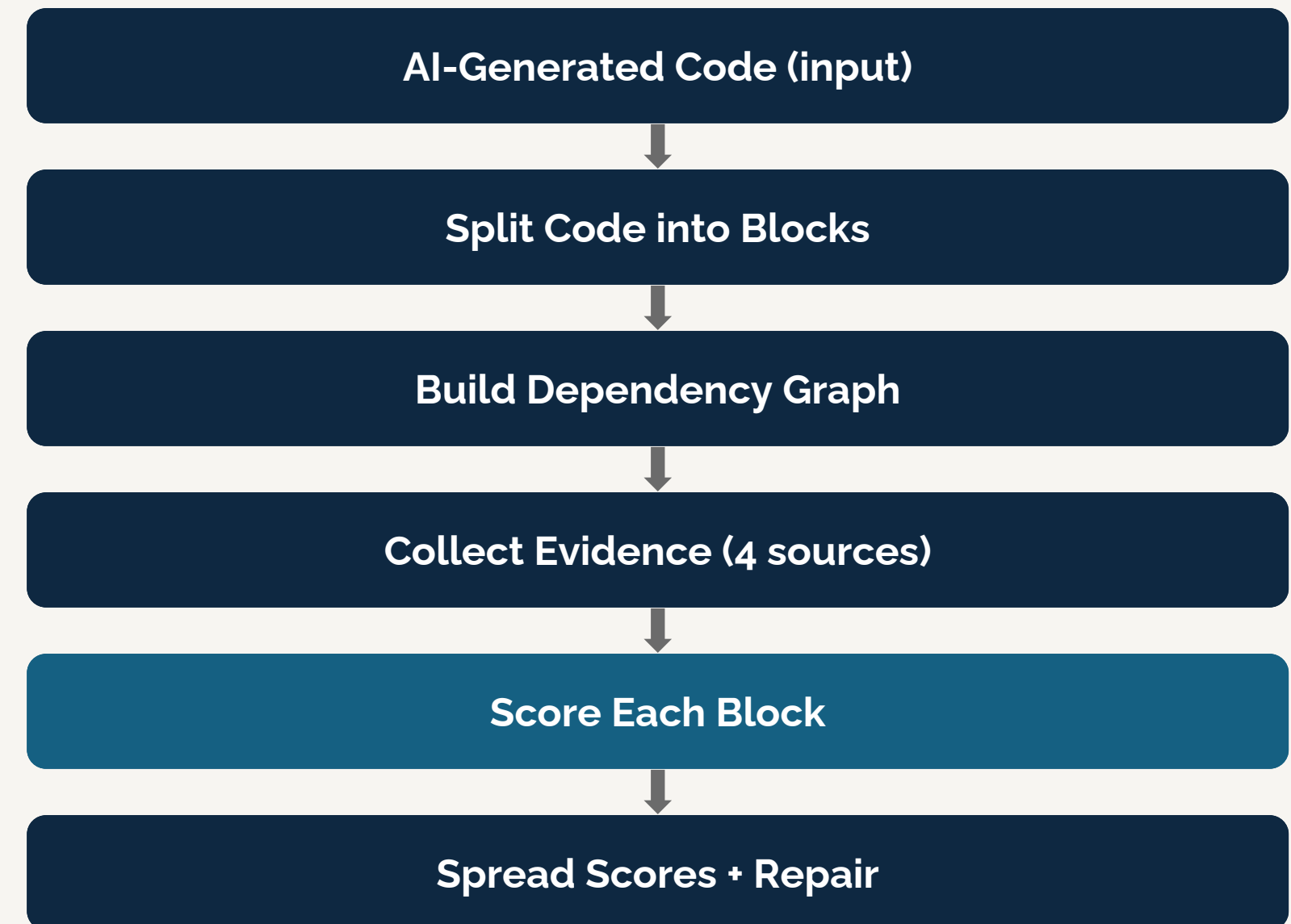
Paper	Focus	Limitation
Sharma & David (2025) arXiv:2502.11620	Uncertainty estimation over a whole AI-generated code answer (entropy / mutual information over sampled generations)	Doesn't localize which part of the code is wrong
Mündler et al. (2025) arXiv:2504.09246	Type-constrained decoding — prevents type errors while the AI is writing code	Doesn't catch logic bugs in well-typed code

Objectives

- **1.** Build a graph linking code blocks that depend on each other.
- **2.** Collect evidence: compiler checks, static analysis, tests, AI confidence.
- **3.** Combine evidence with Bayesian math into one trust score per block.
- **4. Evaluation metric** — precision, recall, F1, and how well-calibrated our probabilities are.

Methodology - Overall

- Code moves through 5 steps — each step feeds the next.
- **Method used:** a Bayesian-style graph. Each code block gets a "how likely is this correct?" score, which can spread to blocks that depend on it.
- The last two steps turn per-block scores into a graph-wide correctness picture, and point to exactly which block to fix.



Methodology – Step by Step

- **Step 1 — AI-Generated Code (input):** candidate program being checked — typically the output of an LLM coding assistant.
- **Step 2 — Split Code into Blocks:** parse the file with Python's AST; each function, method, or class becomes one verifiable unit.
- **Step 3 — Build Dependency Graph:** link blocks by calls, shared data (dataflow), and execution order (sequence) — this is what lets doubt travel between blocks.
- **Step 4 — Collect Evidence:** gather 4 kinds of evidence for every block:
 - Compiler diagnostics — does it parse and run without a syntax error?
 - Static analysis — pylint plus custom checks (unsafe comparisons, mutable defaults, etc.)
 - Execution / tests — do its tests pass; a failing traceback points back to a block.
 - LLM confidence — how “surprised” a small language model is by each line.
- **Step 5 — Score Each Block:** combine the prior belief with all evidence into one probability using Bayesian log-odds fusion.
- **Step 6 — Spread Scores + Repair:** let confidence spread to dependent blocks, then flag the true source for review.

Methodology – PR Inference

- **Type:** approximate inference, not exact — a code graph can have cycles (mutual recursion), so exact enumeration is intractable.
- **Step 1 — Local fusion:** convert the prior belief and every evidence item to log-odds, add them up, convert back to a probability.
 - A passing test or clean lint counts less than a failure of the same kind — a pass is weak proof, a failure is close to proof of a bug.
- **Step 2 — Graph propagation:** a block's final score is capped by how trustworthy the blocks it depends on are, so a broken block pulls down what calls it.
 - Completing this step fully is part of Objective 3 (Bayesian fusion).
- **Evidence trust order:** compiler errors > test failures > static-analysis findings > AI confidence — currently set by hand.

Methodology – PR Learning

- **How the graph is built:** deterministic, not learned — read directly from the code's AST.
 - Each function / method becomes a node.
 - Edges come from 3 relations: calls, dataflow (shared variables), and sequence (execution order).
 - Each edge type carries a fixed coupling strength — how much one block's correctness depends on the other.
- **What is learned:** the numeric trust constants — how much weight each evidence source gets, and how strong each edge coupling should be. Currently set by hand.
- **Plan:** inject known bugs into working code, then tune these trust settings so the model catches them well

Methodology – Decision, Intervention & What-If

- **Decision rule** : a block will become a repair target when its correctness probability drops below a threshold — a direct decision rule over the model's output.
- **Interventional / what-if analysis**: an interactive app lets a reviewer change the evidence-trust weights and propagation strength live and see blocks re-flagged
- This will answer questions like “what if I trusted static analysis less?” — a sensitivity analysis over the model's own parameters.
- **Own vs. inherited doubt** : each block's low score will be split into its own evidence being bad (culpability) vs. something it depends on being bad (inherited).
 - This will answer “if I fixed the dependency first, would this block look fine?” — the intervention question for this project, once propagation is shown.

Methodology – Combining with Modern AI

- The subject of the whole project is AI-written code, so pairing classical reasoning with a real AI model fits naturally.
- We run a small language model over each line of code and measure how “surprised” it is by that line — about 1 second per file.
- We use the single most surprising token, not the average, since that is what usually flags a subtle bug like a flipped comparison.
- **Hybrid:** classic probability math (the graph, log-odds fusion) combined with a real deep-learning signal, working together.

Progress in terms of Project Objectives

- **Obj 1 (Graph) — Done.** Code is broken into blocks and linked correctly.
- **Obj 2 (Evidence) — Done.** All 4 evidence sources are working, including a real AI model.
- **Obj 3 (Bayesian fusion) — Partly done.** Core scoring works now; spreading scores across the graph
- **Obj 4 (Evaluation) — Framework built.** Full results on a bigger test set are coming next review.

Weekly Timeline for Project completion

Week	Milestone
Week 1–2	Literature review + project planning
Week 3–4	Break code into blocks + build the graph
Week 5–6	Build evidence collectors (compiler, static, tests)
Week 7	Add AI-confidence evidence source
Week 8	Bayesian scoring per block
Week 9–10	Spread scores across the graph
Week 11–12	Bug-injection testing + evaluation metrics
Week 13–14	Auto-repair + interactive demo app
Week 15	Final calibration + report

References

- Sharma & David (2025) — [arXiv:2502.11620](#), uncertainty in AI code generation.
- Niels Mündler, Jingxuan He, Hao Wang, Koushik Sen, Dawn Song, Martin Vechev — Type-Constrained Code Generation with Language Models. [arXiv:2504.09246](#).
- Tools: NetworkX, pylint, pytest, HuggingFace Transformers, Streamlit.